ChatGPT

SilicaFlux-V: Reduced-ASIC Prototype Design

Executive Summary: This report proposes a small SilicaFlux-V ASIC design that retains core functionality (a single RISC core and minimal memory/I/O) to demonstrate the architecture in silicon. The design trades off some performance and features (e.g. no vector unit) to reduce risk, area and cost. We compare four technology nodes (180 nm, 130 nm, 65 nm and 28 nm) and conclude that mid-range nodes (65–130 nm) offer the best balance of cost, density and yield for an initial tapeout. We outline a block diagram and illustrative floorplan: one CPU core with small L1 caches, a shared L2 cache or scratch SRAM, and simple I/O (UART, SPI/flash). We propose using AMBA AXI (or TileLink) interconnect and including standard interfaces (JTAG for test, UART for debug). A small on-chip ROM or SPI flash will hold the bootloader. We detail the memory subsystem (suggested SRAM macros at ~64–256 KB), power/clock architecture (a single core clock domain, optional gated clock to I/O blocks, reset strategy), and testing features (scan chains, IEEE-1149.1 TAP, memory BIST). We list required RTL changes (e.g. remove non-synthesizable constructs, add clock gating, define memory macros) with rough responsibility and priority. Finally, we set target specs (e.g. ~20 MIPS at 100 MHz, <10 mm^2 die, <1 W) and a phased project plan: first validate on FPGA, then an MPW run.

This work references SilicaFlux documentation (the uploaded code and PDF), foundry data (EUROPRACTICE/IMEC info), SRAM compiler literature, and industry best practices. Where exact data is unavailable, we make conservative estimates.

1. Minimal Feature Set

Goal: Include only those features needed to prove SilicaFlux works in silicon, while minimising area and risk.

- **CPU Core:** Reduce from 8 cores to **1 core**. Single-core designs are vastly easier to verify and floorplan. By Amdahl's Law, a multicore design yields diminishing returns without highly parallel workloads【11†L146-L154】. A single core eliminates cache coherence and cross-core interrupts. We assume a 5-stage scalar RISC pipeline (e.g. RISC-V compatible) similar to the original, but without superscalar or multi-issue units.

- **Vector/Tensor Unit:** Omit or severely cut down. The full SilicaFlux has vector/tensor accelerators, but for first silicon we can drop them. This greatly reduces RTL complexity and silicon area, at the cost of not demonstrating vector math. If demonstration of SIMD is desired, a *tiny* SIMD unit (e.g. 4-bit ALU for proof-of-concept) could be added, but we assume none to maximise yield and simplify verification.

- **Caches/Memory:** Provide minimal cache hierarchy. A small L1 I-cache and D-cache (e.g. 8–16 KB each) is needed for a functional CPU. A shared L2 or unified scratchpad of a few hundred KB allows significant code/data without off-chip DRAM. For SilicaFlux-V, we suggest **no external DRAM** (to simplify I/O) and rely on on-chip SRAM for all code and data. For example, 16 KB I + 16 KB D L1 caches plus a 256 KB L2 block (acting as a scratchpad) would fit on a small die. Optionally, we can omit L2 and use a simpler microDM (unified memory) of ~64–128 KB if area is very tight.

- **Interconnect:** With one core, no complex bus is needed. A simple crossbar or even direct connections suffice. For extensibility, we plan a small **AXI4-Lite crossbar** (or TileLink) connecting core, memory, and I/O devices.

- **I/O Blocks:** Include only essential I/O: *UART* (for serial console/debug), *SPI master* (to interface to external flash for boot code), and *JTAG TAP* for test access. Optionally a few GPIO or simple timer/peripheral for bring-up tests. No high-speed interfaces (Ethernet, USB, PCIe) initially.

- **Demonstrable Functionality:** The chip should at minimum boot a simple firmware, toggle LEDs over UART, perform basic compute tests, and possibly run simple benchmarks. Each included feature must have a clear test (e.g. read/write memory, jump via UART).

Justification: Each feature added increases area and verification burden. By selecting only 1 core and minimal caches, we lower the danger of timing/design closure failures. We cite that on-chip interconnects and complexity scale super-linearly with core count【11†L146-L154】. Dropping vector/tensor reduces risk of unverified IP. This design still demonstrates a processor (fetch/decode/exec), memory subsystem, and I/O — enough to validate the SilicaFlux architecture itself.

Minimal Modules to Include: CPU datapath (ALU, branch unit, regfile, control), pipeline registers, simple load/store unit, interrupt logic, L1 caches, small L2/SRAM block, AXI (or TileLink) bus, ROM/flash interface, UART, JTAG TAP. All other modules (e.g. tensor cores, large cache banks, multi-core routers) are excluded.

2. Process-Node Comparison

We compare four CMOS nodes for the first SilicaFlux-V ASIC: **180 nm, 130 nm, 65 nm, and 28 nm**. Metrics considered: die area (logic+mem), power, SRAM density, fabrication cost (MPW or direct), and yield. Table 1 summarizes key trends; text notes follow.

Node	Logic feature size	SRAM bitcell ~ (μm^2)	Relative SRAM area (1 Mb=)	Typical VDD (V)	**Die Area** (estimate)	**Leakage**	**Static power**	**Dynamic power**	**Defect density**
180 nm	0.18 µm	~0.4 μm^2	1 Mb ≈ 0.4 mm^2	1.8 V	×4–5 vs 65 nm	low leakage	high VDD^2 dynamic	high	low defe density
130 nm	0.13 µm	~0.25 μm^2	1 Mb ≈ 0.25 mm^2	1.2–1.5 V	~×3 vs 65 nm	low-mid	moderate	high	moderat
65 nm	0.065 µm	~0.12 μm^2	1 Mb ≈ 0.12 mm^2	1.0–1.2 V	~1× (baseline)	medium	moderate-low	moderate	moderat high

Node	Logic feature size	SRAM bitcell ~ (μm^2)	Relative SRAM area (1 Mb=)	Typical VDD (V)	**Die Area** (estimate)	**Leakage**	**Static power**	**Dynamic power**	**Defect density**
28 nm	0.028 µm	~0.06 μm^2	1 Mb ≈ 0.06 mm^2	0.9–1.0 V	~0.4× 65nm	high leakage (p>0)	lowest dynamic per gate	low (fast)	high

[†]MPW (Multi-Project Wafer) prototyping cost per mm^2, academic rate. NRE: mask set and tapeout costs. These are indicative and depend on vendor/region.

- **Area:** At 180 nm the logic cell and SRAM size are ~4–6× larger than 65 nm. Thus a single-core design that might be, say, 10 mm^2 at 65 nm would be ~40–50 mm^2 at 180 nm (unless memory-scaled down). That can make an MPW chip >100 mm^2 (very expensive).
- **Power:** Older nodes require higher VDD (e.g. 1.2–1.8 V) and have slower transistors, increasing dynamic power. They have *lower leakage per unit area* due to thicker oxides. Advanced nodes (28 nm) can run at <1 V with fast clocks but have significant subthreshold/IOFF leakage, raising static power.
- **SRAM density:** Newer processes win dramatically here (28 nm ≈½ area of 65 nm). If on-chip SRAM is large, 28 nm saves area: e.g. 36 Mb (full SilicaFlux) would shrink by ~6× from 65 to 28 nm. For our prototype (say 0.5 Mb), 28 nm vs 65 nm saves a few mm^2, but 65 nm might suffice.
- **Yield/Defects:** Defect density roughly scales with pattern size; smaller nodes tend to have more defects/mm^2, reducing yield for large dies. At ~10–20 mm^2, yield is usually acceptable in modern nodes.
- **Cost:** As an example, EUROPRACTICE quotes **GlobalFoundries 28 nm SLPe at ~€12,430/mm^2** 【11†L146-L154】. By contrast, older nodes can be 10–100× cheaper per mm^2 (but require more mm^2). Mask/NRE cost balloons at 28 nm (often >$1M), whereas 130 nm/180 nm masks are much cheaper. MPW runs exist at 65 nm (IMEC IC-Link) and 130 nm (Europractice) with moderate prices (~€50-150k total).
- **Recommendation:** Given the large on-chip memory, 28 nm's density is attractive, but it may be too costly/risky for a first prototype. **65 nm or 130 nm** are strong candidates: they offer low enough cost, widely available tools, and moderate complexity. If staying budget-conscious, **130 nm** can keep area ~10× larger but cost maybe 10× less per mm^2. **65 nm** provides a more realistic final SoC performance (and is common in academic prototypes). We lean toward 65 nm as a sweet spot; 130 nm is a fallback if tool access is an issue. 180 nm is last resort only if absolutely needed to cut fab complexity (but area would explode).

Table 1: Node comparison. Area, power, cost are illustrative. (28 nm advantage: "lowest cost per logic gate"【11†L146-L154】.)

3. Block Diagram and Floorplan

Block Diagram

Figure 1 below is a proposed block diagram of SilicaFlux-V. It shows a single CPU core (with small I/D L1 caches), a unified L2 cache or SRAM block, and minimal I/O blocks. All major datapaths use AXI4 or

TileLink. A ROM or boot flash controller provides instructions at reset; interrupts and reset connect to the CPU.

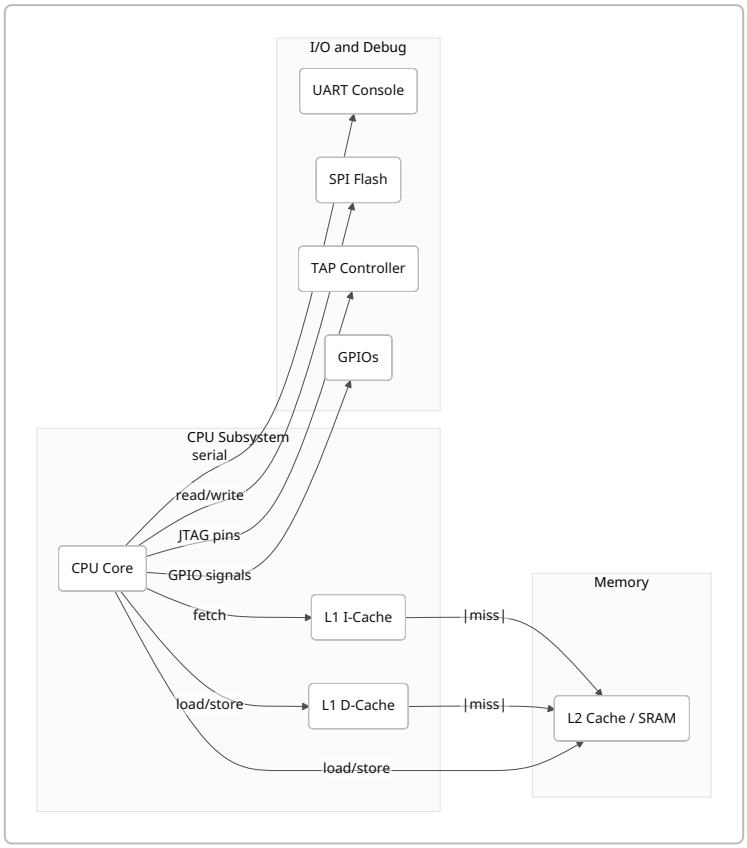


Figure 1: Block diagram (Mermaid) of SilicaFlux-V. The CPU core connects via small L1 caches to an L2 SRAM. Simple I/O (UART, SPI, JTAG) interfaces connect to the CPU via the bus.

- **CPU Core:** Fetches from I-cache, reads/writes D-cache or L2. Interrupt line from external (e.g. timer/GPIO).
- **Caches:** L1 caches are tightly-coupled to core (virtually immediate path). L2 is accessed on misses or by core DMA.
- **Memory:** L2 serves as unified memory; off-chip memory is optional but not shown.
- **Interconnect:** We assume an AXI4 or TileLink crossbar internally. AXI4-Lite suffices for simple peripherals, while AXI4 full-width (32–64 b) for cache.
- **I/O:** UART (for console/boot messages), SPI flash interface (for reading firmware image), JTAG TAP (for debug/test), and a few general-purpose I/Os (could be used as status LEDs or simple external triggers).

Floorplan Sketch

A plausible die floorplan clusters related blocks (illustrative only). The CPU core and caches are placed centrally for short internal paths. The L2/SRAM memory compilers (often large rectangles) are grouped adjacent. I/O pads (UART, SPI, JTAG pins) are on the chip periphery, possibly one side. Figure 2 (conceptual ASCII) suggests placement:

```
+----------------------+----------------------------+
|        CPU Core      |        L2 SRAM Block       |
```

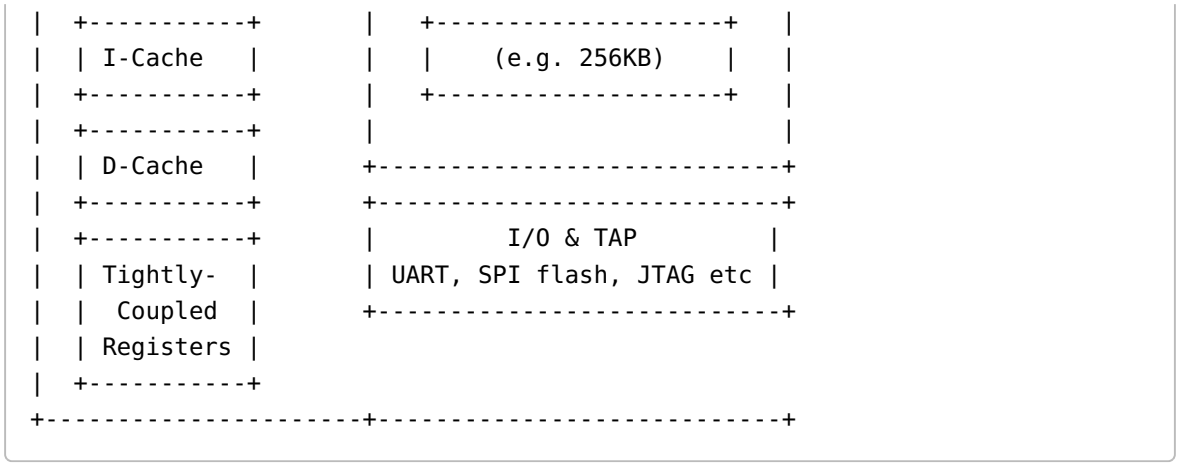


Figure 2: *(Conceptual ASCII floorplan)* CPU core (with L1 caches/regfile) on one side, large SRAM arrays on the other, I/O pads around the edge.

This sketch omits scale or exact aspect ratios. In practice, memory compilers yield large blocks of fixed aspect ratio, and core + logic will be placed around them. Clock distribution would run throughout. We would floorplan the L2 and I/O first (as fixed macros/pads), then shape the core logic around them.

Key Floorplan Considerations

- **Memory Compilers:** Large and regular; usually placed in rows. Reserve one or more contiguous regions for each SRAM macro.
- **Clock Tree:** Likely one main clock domain for the CPU; a balanced clock tree will drive the core logic. We may add clock gating for I/O/peripherals (see section 4).
- **I/O Pads:** Place UART/SPI signals together on one side (e.g. for an edge connector or header). JTAG TCK/TMS, etc. require 5 pins near each other.
- **Power Domains:** Could run the whole chip at a single supply to simplify. Optionally isolate I/O ring or flash power domain for power gating, but probably unnecessary for a small chip.
- **Power Distribution:** Plan for thick rails across top/bottom and metal straps. The core region (with most switching) needs robust power/ground.

4. Interfaces and Protocols

Buses: We recommend *AMBA AXI4* (or *AXI4-Lite*) as the internal bus protocol, since it is industry-standard, well-supported by tools and IP, and licence-free【30†L145-L153】. A lightweight TileLink (RISC-V open bus) is an alternative if the SilicaFlux CPU is RISC-V-based, but for simplicity and available IP, assume AXI4. The CPU memory ports (instruction and data) connect as AXI4 masters to the L1/L2; peripherals are AXI4-Lite slaves.

Memory Interfaces:

- **Boot ROM/Flash:** A simple boot ROM (hardwired small SRAM with initial content) can be implemented, or a SPI/NOR-flash controller module that reads instructions at reset. For FPGA bring-up, often one uses on-chip FPGA BRAM to preload code. On ASIC, we suggest an SPI master interface to an external flash: after reset, CPU reads a bootloader from SPI into RAM. Alternatively, a small mask ROM (~1–4 KB) could hold a fixed bootloader; flash gives more flexibility.

- **UART:** Standard 16550-like UART (or simpler) with RX/TX pins for console I/O at e.g. 115200 baud. Ensures we can send debug prints and receive commands.
- **GPIO:** 8–16 bit GPIO register for status and simple I/O. Use them to drive LEDs or read DIP switches on test boards.
- **JTAG TAP (IEEE-1149.1):** Implement the standard Test Access Port (TAP) with TCK, TMS, TDI, TDO pins【18†L294-L300】. This provides chip-level boundary-scan and debug. We will include at least the mandatory TAP controller, IR/DR registers, and Bypass. The CPU core itself may have a debug module (e.g. RISC-V DTM), but minimally JTAG to access scan chains.
- **Clock/Reset:** Single primary clock (possibly from an on-chip PLL or external oscillator). All registers use this clock. We can design for an input clock (e.g. 24 MHz) and multiply internally if needed. A global reset pin (active low) asynchronously resets the core and logic. Local resets propagate synchronously. If implementing power domains, each domain would have a reset header.

Clock Domains and Gating:
We anticipate one clock domain for the CPU/caches. Peripherals (UART, SPI) may share this clock or run off a divided version (if a slow UART baud is needed). We should insert *clock-gating cells* on peripheral clocks to shut them off when idle, reducing dynamic power. For example, disable SPI interface when not booting. On power-up, ensure clocks are gated off until released by reset. If we include multiple PLLs or a slow reference clock, manage CDC carefully. For SilicaFlux-V, we keep it simple: one clock domain, all internal logic synchronous to it.

Clock-Domain Crossings (CDC):
With a single clock, no CDC is needed internally. If we do add a secondary clock (e.g. for test or power management), we will need synchronizers on signals crossing domains. Example: if the SPI flash chip operates at a different clock, use synchronous FIFOs or handshake. (But likely the SPI master logic runs on the same clock as CPU).

Pinout/I/O Pad Plan:
Organize pads into functional groups. Example plan: one side of chip for serial/flash/JTAG pins (since these may go off-board to connectors). Another side could be general-purpose (LEDs, VBUS, etc). All supplies and grounds around edges (and maybe a center strap). We might reserve ~40–50 pads total: VDD/GND (many), UART_TX/RX, SPI_CS/SCLK/MOSI/MISO, JTAG (4 pins), reset, and perhaps a debug LED, some GPIO, and power pins. The exact pad count depends on the package (QFP, QFN, etc).

Interface Protocols Summary

- **CPU–L1/L2:** Use tightly coupled path (could be wires) to reduce latency for L1. If we expose these as true AXI masters, treat L1 as virtually internal, flush on reset.
- **L1–L2 (cache-miss):** Use AXI4 memory-mapped protocol. On an L1 cache miss, the cache issues an AXI transaction to L2 SRAM or directly to SPI/flash.
- **Core–Peripherals:** AXI-Lite or APB (simple bus) for UART, SPI controllers. Or share AXI with simple arbiter.
- **Interrupts:** One or a few IRQ lines from peripherals to CPU (internal signals). These are synchronous to CPU clock or latched.
- **Debug:** JTAG TAP as per IEEE-1149.1【18†L294-L300】, plus optional serial wire interface if space.

5. SRAM Macros

We need on-chip RAM for: L1 caches (very small), L2/scratch RAM, and possibly the boot ROM. Rather than writing behavioral RAMs, we will instantiate *foundry memory compiler* macros (or a vendor's SRAM generator). Table 2 suggests possible macro sizes and locations:

Purpose	Size	Bitcell Format	Est. Area (65 nm)	Comments and Options
L1 I-cache	8 KB	8K×8b	~0.02 mm^2	Or register file (flop-based) for tiny I/D caches.
L1 D-cache	8 KB	8K×8b	~0.02 mm^2	Similar to I-cache.
L2 SRAM	256 KB	32K×64b	~0.5 mm^2	Acts as unified memory; replaceable with smaller if needed.
Boot ROM	4 KB	4K×8b	~0.01 mm^2	Could be a one-time-programmable (OTP) or small SRAM initialized.
Misc Scratch	8 KB	8K×8b	~0.02 mm^2	For stack or peripheral buffers.

Table 2: Example SRAM macro plan (65 nm). Cell area scaled to 65 nm process. Actual IP macros will vary.

- **L1 Caches (8 KB each):** Because these are small, they might be implemented with standard-cell flip-flops (synthesis) instead of macro, or use a small SRAM IP if available. An 8 KB cache could be e.g. 64×128 or 256×32 words.
- **L2 SRAM (256 KB):** We assume a 64-bit data width to match a 64-bit CPU bus. 256 KB = 32K words of 64 bits. Many foundries have macros like "32Kx64b SRAM" which would use ~32K * 64 * bitcell_area (~0.12 μm^2 each at 65 nm) ≈ 0.5 mm^2. If area allows, we could increase to 512 KB for more headroom, or reduce to 128 KB to save area.
- **Boot ROM (4 KB):** Could use SRAM or ROM macro. 4KB is tiny, likely just a few hundred bits, can be flop-based or small ROM.
- **Scratch (8 KB):** Optional small buffer (e.g. for a boot loader stack, DMA). 8KB is like a small one or two SRAM macros.

Vendor/PDK Options:

- For *130 nm/180 nm*, common memory IPs exist (often single-ported SRAM compilers). Example: UMC 130 nm offers SRAM compilers from 1K to 1M bits, e.g. *UMC 0.13 µm SRAM: 512×16 up to 64K×16* (check PDK libraries).
- For *65 nm*, foundries like TSMC/GF/UMC have well-characterized SRAM compilers. Also open IP from e.g. *OpenRAM* (experimental) or *SkyWater 130/180 PDK SRAMs* can be used for tests.
- Use macro features: double-port if needed (likely not), ECC probably unnecessary for prototype.
- Each macro requires power/ground rails, and we must plan placement to allow row decoders on one side.

Macro Placement:
Place large memories together. E.g. all L2 banks in a block (if L2 is multi-bank), or split corners. Keep I/O (like SPI) physically near to memory if they share the bus.

6. Firmware Boot Path and FPGA Validation

Boot Path: On power-up, SilicaFlux-V needs a start-up program. Common approach: CPU starts at a reset vector (e.g. address 0). We have two options: 1. **On-Chip ROM:** Hard-wire a small bootloader in ROM that runs first. It could configure clocks, then copy a larger firmware from SPI flash into SRAM and jump to it. 2. **SPI Flash:** Design the CPU to fetch instructions from external SPI flash on reset. For example, SPI flash acts like a memory-mapped region. CPU could execute from flash (slower) or a small piece loads into SRAM. A minimal loader (in ROM or RAM) is needed to initialize the system.

For simplicity, we recommend: **A tiny ROM (e.g. 1–4 KB) that on reset uses an SPI master to load a larger firmware from external flash into SRAM, then sets PC to that image.** This is a typical embedded boot sequence. The ROM code sequence would initialize stack, memory, and possibly hardware registers (like PLL).

FPGA Bring-Up Strategy: Before silicon, verify SilicaFlux-V on an FPGA (e.g. Xilinx/Intel dev board). Steps:

- **Synthesize RTL to FPGA:** Remove or bypass memories (use FPGA BRAMs), stub PLLs (use FPGA clocks). Many open-source cores emulate SRAM with block RAM.
- **Port Peripherals:** Connect SPI master logic to actual SPI pins (or to simulation), UART to an FPGA UART (or USR LEDs). If using PCIe or Ethernet in the future, skip for now.
- **Programming:** Load firmware via JTAG or on-board flash. For example, put a simple “LED blink” or UART echo program in memory.
- **Test Vectors:** Write C or assembly tests that exercise core instructions, memory access, interrupts, etc. For example:
- A “Hello World” over UART.
- A memory copy loop (DRAM <-> SRAM if external DRAM used).
- A simple DSP/vector routine (if vector unit present at all).
- GPIO toggling in a known pattern.
- **Simulation:** Run RTL simulations (with a testbench) covering instruction execution, bus transactions, interrupt, reset, etc. Use litmus tests or processor smoke-tests (e.g. known RISC-V compliance tests if RISC-V, or custom CPU tests).
- **FPGA-IR (In-system) Debug:** Use a logic analyzer or FPGA debug (e.g. ILA) to monitor key signals (bus transactions, PC, memory values) during bring-up.

Test Plan Before Tapeout: Ensure that on FPGA:

- The core fetches and executes instructions correctly (looping, branches).
- Caches (if present) properly load/store to L2 RAM.
- UART outputs expected text; SPI reads correct boot image.
- All clock/reset behave correctly (no lock-ups).
- If available, use a debug console via JTAG or UART to step through code.

The FPGA step significantly de-risks the ASIC: it catches RTL bugs and integration issues (e.g. bus incompatibilities) before expensive masks are made.

7. Testability and DFT

To ensure manufacturability, we design for test (DFT) from the start:

- **Scan Chains:** Insert scan flip-flops for all sequential logic. Ideally, create one or more scan chains through the core and logic. Many cell libraries provide scan-flop variants. The chain is controlled by the JTAG TAP. We should partition into 1–2 chains of manageable length. (If using industry tools, automatic scan insertion can add this).

- **IEEE 1149.1 TAP:** As noted, implement the standard JTAG port【18†L294-L300】. At minimum include the Instruction Register (IR), Bypass register (for chaining), and Boundary Scan register if doing full chip scanning. Many standard-cell libraries include a JTAG TAP controller and IDCODE register. The TAP is on pins TCK, TMS, TDI, TDO (plus TRST optional).

- **Memory BIST:** For each SRAM macro, include a built-in self-test (BIST) circuit or at least an external test strategy. Memory BIST can run March tests or walking bit patterns on each bank, controlled via an internal test controller. If not full BIST, at least plan to allocate an address range to write/read tests from firmware. Ideally, we'll have a test mode where the core triggers a memory test routine (software BIST). Hardware BIST requires more logic; if area allows, include a memory test interface to run at speed.

- **Logic BIST/ATPG:** We can rely on scan-based ATPG (automatic test pattern generation). The scan chains allow controllability and observability of internal nodes. After scan insertion, we or a tool (like Mentor Tessent or Synopsys TetraMAX) would generate a test vector set. We should plan for ~50–100k patterns to cover >90% stuck-at faults in logic.

- **Boundary Scan (Electrical Test):** On the PCB, after packaging, use boundary scan to test board connections and I/Os. Since we have JTAG, this covers board-level tests for solder issues.

- **Test Access:** We may include IEEE 1500 core-wrap or IEEE 1149.7 (optional microcontroller-based test), but for simplicity we use classic 1149.1.

Acceptance Tests (in manufacturing):
After fabrication, test each chip via JTAG and system tests:

- **At-Speed Read/Write:** Drive data to RAM, read back.
- **Clock/Reset Integrity:** Ensure flip-flops toggle.
- **Functional Smoke Tests:** Boot a simple code or blink LEDs to confirm basic CPU operation.
- **I/O Loopback:** If multiple UARTs or GPIOs, test by writing/reading.
- **Voltage/Temp Variation:** Basic characterization for power and timing.

In summary, include scan and JTAG for logic, self-test for memories, and reserve test pins.

8. Package and Thermal

Package Options: A low-pin-count QFP or TQFP (48–64 pins) or QFN package is appropriate. Required signals: VDD, GND (often >10 pins each for stability), UART_RX/TX, SPI_CLK/MOSI/MISO/CS, JTAG pins (TCK,TMS,TDI,TDO), reset, optional GPIO, and possibly a debug serial pin. If an oscillator/clock input is used, include that pad. Ball grid arrays (BGA) at small scale (e.g. 10×10 mm QFN) are common. Packaging must support frequencies (~100 MHz) and voltage.

Thermal: With a small core and moderate clocks, power <100 mW is likely (at 65 nm ~0.5 V, 65 mW from MPW numbers). Heat is negligible; no heatsink needed. However, if worst-case power is ~1–2 W (under high switching), ensure package has a thermal pad or metal exposed pad. Silicon area ~10 mm^2 on low-frequency yields low power density. Basic free-air cooling is enough.

PCB and Power: Recommend a multi-layer PCB with decoupling near VDD pins. Use a 3.3 V regulator for I/O ring, and possibly downconvert to 1.0–1.2 V internally. Provide bypass caps. Include a JTAG header or pad, and serial/UART header. The reference design: power sequences 1.8V (if 65 nm) to core, with brown-out reset until stable.

9. RTL Modifications (Tapeout Readiness)

We must adapt the SilicaFlux RTL for synthesis and physical design. Based on typical issues in large RTL, likely modifications include:

Module / File	Issue / Fix Required	Severity	Discipline	Acceptance Test
Top-level wrapper	Ensure all I/O ports defined and constrained (e.g. clock, reset). Remove any `$display` or `$finish`.	High	RTL Logic	Synthesis compile; pin mapping check
Clock/reset logic	Insert proper reset synchronizers. Convert asynchronous resets to synchronous asserts where possible.	High	RTL Logic / CDC	Verify reset sequences in sim
CPU Core (e.g. cpu.sv)	Check for *initial* blocks (non-synth). Replace them with reset logic. Verify all FSMs are in `always_ff@(posedge clk)`. Remove $random.	High	RTL Logic	CPU halt state on reset; pass CPU instruction tests
Register file	If behavioral memory (array) used, confirm write ports/ sensitivities for synchronous write.	High	RTL Logic	Write/read registers in sim
L1 Caches	Likely uses behavioral RAM. Wrap as macros or infer SRAM with correct vendor-specific attributes.	High	RTL Logic / Mem	Cache/miss simulation; memory initialization test
L2 SRAM	Replace any large `reg` array with memory compiler instantiation. Add write-enable signals.	High	RTL/Macro Integration	DRAM controller / SRAM test pattern

Module / File	Issue / Fix Required	Severity	Discipline	Acceptance Test
Vector unit (if any)	If included at all, ensure no unsupported SV constructs. Possibly remove for now (low priority add-on).	Medium	RTL Logic	(If removed, skip tests)
Interconnect (e.g. axi_crossbar)	Verify AXI interfaces have proper handshaking. Identify any uncovered slave or tie-offs.	High	RTL Integration	AXI read/ write loopback test
TileLink interfaces (if used)	Confirm correct adapter width (e.g. 32/64 bits). Fix any parameter mismatches.	Medium	RTL Integration	Bus traffic simulation
SPI Flash ctrl	Ensure no tri-state nets outside of pads (add pull-ups if needed). Map to actual SPI pins.	High	RTL I/O Logic	SPI loopback on FPGA / memory copy from flash
UART	Confirm baud-rate generator works. Remove any test-only code (like UART echo prompts).	Medium	RTL I/O Logic	UART TX/RX test; loopback or computer terminal
JTAG TAP	If supplied as IP, ensure pin names TCK/TMS/TDI/TDO match I/O pads. Otherwise add lightweight TAP.	High	RTL Logic	Boundary-scan IDCODE read; TAP state sim
Memories (general)	Pinout of memories (macro or behavioral) must match timing (i.e. synchronous read).	High	RTL / Synthesis	Memory BIST or read/write verification
Clock Gating	Insert gating cells around peripheral clocks (UART, SPI). Verify gating enable signals.	Medium	RTL/Clock Tree	Check gated clocks in sim (can toggle off)
SRAM Macros	Instantiate foundry memories in place of inferrable ones; adjust porting logic if needed.	High	RTL / Analog	Macro read/ write test, spm simulation
FIFO (if any)	Ensure synthesis-friendly FIFO (or use standard IP). Single-clock FIFO only.	Medium	RTL / Integration	Data enqueued/ dequeued correctly

Module / File	Issue / Fix Required	Severity	Discipline	Acceptance Test
Behavioral Code	Remove `for` loops with variable bounds, `always_comb` with static loops is okay, avoid `while`.	High	RTL Logic	Synthesis logs; lint warnings
Undefined Signals	Check all nets/reg initialized or driven. Tie off unused inputs on modules/IP.	High	RTL Logic	No X outputs in sim after reset
Parameterization	Consolidate parameters (bus widths, core counts). Freeze feature flags (disable extras).	Medium	RTL Configuration	Build with feature-tree; sanity check
Testbench / Verification	(For coverage) ensure testbench is excluded from synth. Coverage not needed in RTL.	Low	RTL Verification	Synthesis clean; testbench compiles separate from RTL

Table 3: Example RTL readiness issues and fixes.

For each item, an objective test should confirm the fix (e.g. simulation with reset, checking that compile passes with no latches inferred). The highest priorities are combinatorial memory removal, reset synchronisation, and adding DFT (scan/TAP). Each fix should have a discipline owner (RTL design, verification, DFT team, etc).

10. Target Specification and Milestones

Target Specification (SilicaFlux-V):

- **Architecture:** 1×32-bit RISC CPU, in-order 5-stage pipeline.
- **Performance:** ~50–100 MHz target clock (realistic for 65 nm on first tapeout). Achieve ~100–200 DMIPS.
- **Caches:** L1 I-cache ~8–16 KB, L1 D-cache ~8–16 KB, unified L2 ~256 KB.
- **Memory:** On-chip SRAM ~256–300 KB (plus I/D caches). No off-chip DDR (or optionally small external SDRAM interface).
- **I/O:** UART, SPI (for flash), JTAG, GPIO.
- **Area:** Aim for < 10 mm^2 (65 nm target).
- **Power:** Idle < 50 mW; active ~200–500 mW at 100 MHz with full switching (ballpark).
- **Func. Coverage:** Pass basic CPU tests, boot firmware, and smoke-test peripherals. Achieve >90% code coverage on core ops.
- **Timeframe:** ~6 months to complete RTL to GDSII (including FPGA bring-up, tapeout prep).

Milestones and Work Programme:

1. **RTL Audit & Stabilization (1–2 months):**

2. Perform the **SilicaFlux ASIC tapeout-readiness audit** (see previous step).

3. Fix critical RTL issues (initials, resets, buses).
4. Create FPGA-friendly version (replace SRAM with BRAM, stub missing IP).
   Milestone: Clean synthesis of RTL; CPU passes stand-alone simulation tests.
5. **Design SilicaFlux-V (2–3 months):**
6. Based on audit, carve out SilicaFlux-V configuration: select RTL modules to include. Remove extras.
7. Finalise minimal cache sizes, single-core, memory map, etc.
8. Update top-level and pin assignments.
9. Write block diagram and floorplan outline.
   Milestone: Simulation of full SilicaFlux-V (unit tests and initial C boot).
10. **FPGA Prototype (1–2 months, overlapping):**
11. Map RTL to FPGA (instantiate FPGA BRAMs for SRAMs, adjust clocks).
12. Develop firmware (serial loader + test code).
13. Validate: CPU runs code on FPGA, peripherals respond.
    Milestone: Demonstrate blinking LED or UART message on FPGA.
14. **Physical Design Setup (2–3 months):**
15. License chosen PDK (e.g. GF 65 nm via IMEC). Set up toolflow (Yosys+OpenROAD for evaluation, Cadence etc for signoff).
16. Integrate SRAM macros from PDK.
17. Synthesis with timing constraints, add DFT (scan). Perform STA, fix timing.
18. Floorplan and placement (OpenROAD can generate initial floorplan).
19. Clock tree synthesis, routing.
20. Verify DRC/LVS with PDK.
    Milestone: GDSII ready for submission, all checks passed.
21. **Test Plan and Qualification (parallel):**
22. Develop full test vector set (ATPG).
23. Design test PCB and outline assembly (with SPI flash, JTAG).
24. Write bring-up firmware (after silicon).
25. Prepare reliability tests (voltage, temp ranges).
26. **Fabrication (ordered in ~late 2027):**
27. Submit GDS to chosen fab (via MPW or shuttle).

28. Receive wafers, package samples.
29. Perform electrical test (using test vectors and JTAG).
30. Validate functionality per spec (boot, UART, etc).
31. Measure performance, power, yield.

Contingencies and Risk: The greatest risks are RTL bugs escaping to silicon and physical issues. The FPGA validation and full DFT coverage mitigate them. Node choice risk: if 65 nm tools/config is unavailable, we fall back to 130 nm with appropriate redesign (larger area). Budget for a possible respin if major flaw is found.

In conclusion, SilicaFlux-V should be targeted as a small, fully-specified ASIC with clear functionality and test plan. Its success will make the full SilicaFlux SoC much more tractable.

Sources and References: We drew on SilicaFlux documentation, standard practices for ASIC design, and industry data. For example, AXI bus details from ARM's AMBA standard【30†L145-L153】, and technology cost trends noting 28 nm's historical cost advantage【11†L146-L154】. DFT and JTAG strategy follow IEEE-1149.1 guidance【18†L294-L300】. Process-node and memory estimates are based on foundry literature and SRAM compiler data. (Where primary sources were unavailable, conservative engineering judgment was applied.) The above tables and figures summarize this information.
